

Zero-Trust-Sicherheitskonzept mit GitHub-Integration

Einführung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-basierter Workflow-Integration

Prüfungsfach:	Betriebliche IT-Prozesse	
Fortbildungsprüfung:	Certified IT Business Manager (IHK)	
Prüfling:	Daniel Massa, Prüflingsnummer 615951	
Projektbetrieb:	Verein zur Förderung der Berufsbildung e.V., Kurfürstenstraße 6, 71636 Ludwigsburg	Erklärung:
Wohnanschrift:	Hackstraße 41, 70190 Stuttgart	
Abgabedatum:	01.11.2026	
Betreuer im Betrieb:	Carsten Vordermeier	
IHK-Betreuer:	Frau Dr. Sabine Wagner, IHK Stuttgart	

Die Projektarbeit wurde selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt. Stuttgart, den 01.11.2026 _____

Daniel Massa

Contents

1	Zero-Trust-RBAC-Pilot – Projektdokumentation	iii
1.1	0 Management Summary	iii
1.2	1 Projektinformationen und Initiierung	iii
1.2.1	1.1 Projektumfeld	iii
1.2.2	1.2 Ausgangslage	iii
1.2.3	1.3 Problemstellung	iv
1.2.4	1.4 Projektziel	iv
1.2.5	1.5 Projektabgrenzung	iv
1.2.6	1.6 Projektauftrag	iv
1.2.7	1.7 Persönliche Projektrolle und Eigenleistung	iv
1.3	2 Projektplanung und Projektmanagement	v
1.3.1	2.1 Vorgehensmodell	v
1.3.2	2.2 Projektstrukturplan	v
1.3.3	2.3 Zeitplanung	v
1.3.4	2.4 Ressourcenplanung	vi
1.3.5	2.5 Kostenplanung	vi
1.3.6	2.6 Stakeholder	vi
1.3.7	2.7 Kommunikation	vii
1.3.8	2.8 Risikomanagement	vii
1.3.9	2.9 Qualitätsmanagement	vii
1.3.10	2.10 Meilensteine und Controlling	vii
1.3.11	2.11 Entscheidungs- und Änderungsmanagement	viii
1.4	3 Analyse und Lösungsentscheidung	viii
1.4.1	3.1 Ist-Analyse	viii
1.4.2	3.2 Anforderungen	viii
1.4.3	3.3 Lösungsalternativen	viii
1.4.4	3.4 Nutzwertanalyse	viii
1.4.5	3.5 Entscheidung	ix
1.4.6	3.6 Datenschutz	ix
1.4.7	3.7 Sicherheitsanforderungen	ix
1.4.8	3.8 Abnahmekriterien	ix
1.5	4 Technische Konzeption	ix
1.5.1	4.1 Zielarchitektur	ix
1.5.2	4.2 Rollenmodell	x
1.5.3	4.3 Status- und Genehmigungsworkflow	x
1.5.4	4.4 Datenmodell	x
1.5.5	4.5 API	xi
1.5.6	4.6 Policy-Prüfung	xi
1.5.7	4.7 GitHub-Integration	xi
1.5.8	4.8 Audit-Hash-Kette	xii
1.5.9	4.9 Fehler-, Retry- und Wiederanlaufkonzept	xii
1.5.10	4.10 Deployment, Logging und Monitoring	xii
1.6	5 Durchführung	xii
1.6.1	5.1 Ausgangsstand	xii
1.6.2	5.2 Backend-Implementierung	xiii
1.6.3	5.3 Datenhaltung	xiii
1.6.4	5.4 Rollen- und Policy-Logik	xiii
1.6.5	5.5 Genehmigungsworkflow	xiii
1.6.6	5.6 Provisionierungs-Dry-Run	xiii

1.6.7	5.7 Auditverkettung	xiii
1.6.8	5.8 CI und Security	xiii
1.6.9	5.9 Abweichungen und Entscheidungen	xiv
1.6.10	5.10 Persönliche Eigenleistung	xiv
1.7	6 Test und Qualitätssicherung	xiv
1.7.1	6.1 Teststrategie	xiv
1.7.2	6.2 Testumgebung	xiv
1.7.3	6.3 Testfälle	xiv
1.7.4	6.4 Testergebnis	xv
1.7.5	6.5 Fehleranalyse	xv
1.7.6	6.6 Korrektur und Retest	xv
1.7.7	6.7 Security-Ergebnisse	xv
1.7.8	6.8 Traceability	xv
1.8	7 Übergabe und Abnahme	xvi
1.8.1	7.1 Betriebsdokumentation	xvi
1.8.2	7.2 Benutzer- und Testanleitung	xvi
1.8.3	7.3 Übergabepaket	xvi
1.8.4	7.4 Abnahmekriterien	xvi
1.8.5	7.5 Tatsächlicher Abnahmestatus	xvi
1.8.6	7.6 Restpunkte	xvi
1.9	8 Projektabschluss	xvii
1.9.1	8.1 Projektergebnis	xvii
1.9.2	8.2 Soll-Ist-Vergleich	xvii
1.9.3	8.3 Zeitbewertung	xvii
1.9.4	8.4 Kostenbewertung	xvii
1.9.5	8.5 Qualitätsbewertung	xvii
1.9.6	8.6 Nutzen	xvii
1.9.7	8.7 Restrisiken	xviii
1.9.8	8.8 Lessons Learned	xviii
1.9.9	8.9 Persönliches Fazit und Ausblick	xviii
1.10	9 Quellen und Hilfsmittel	xviii
1.10.1	Quellen	xviii
1.10.2	Hilfsmittel (KI)	xviii
1.11	Anhang	xviii
1.11.1	A: Testprotokoll (Auszug)	xviii
1.11.2	B: Security-Reports	xviii
1.11.3	C: Claim-Evidence-Matrix	xix
1.11.4	D: Sign-Off-Paket	xix
1.11.5	E: Fehler- und Retest-Dokumentation	xix
1.11.6	F: KI-Transparenzdokumentation	xix

1 Zero-Trust-RBAC-Pilot – Projektdokumentation

Projekt: Einführung eines Zero-Trust-Sicherheitskonzepts mit GitHub-basierter Workflow-Integration

Autor: Daniel Massa

Stand: 10.07.2026

Status: Technische Vorbereitung abgeschlossen – Persönliche und betriebliche Bestätigung ausstehend

1.1 0 Management Summary

Gegenstand dieser Projektarbeit ist die Konzeption und prototypische Umsetzung eines Zero-Trust-Rollenkonzepts für die Rechteverwaltung in GitHub-Organisationen.

Im Rahmen einer technischen Vorbereitung (08.–10.07.2026) wurde ein voll funktionsfähiger FastAPI-Pilot mit folgenden Eigenschaften erstellt:

- **16 API-Endpunkte** für Benutzer-, Rollen- und Zugriffsverwaltung
- **Sechs Pilotrollen** mit abgestuften Risikostufen
- **Policy-gestützter Genehmigungsworkflow** mit Statusautomat (DRAFT → SUBMITTED → APPROVED/REJECTED → PROVISIONED/FAILED)
- **SHA-256-Audit-Hash-Kette** mit Manipulationserkennung
- **GitHub-Dry-Run-Provisionierung** (Simulation, keine produktive Änderung)
- **14 automatisierte Testfälle**, vollständig bestanden
- **CI/CD-Konfiguration** und **Security-Scans**

Die Dokumentation beschreibt den realen Ist-Stand des Pilotprojekts. Offene Punkte (Auftraggeberbestätigung, Abnahme, persönliche Unterschrift) sind als solche gekennzeichnet und liegen in der Verantwortung des Autors.

1.2 1 Projektinformationen und Initiierung

1.2.1 1.1 Projektumfeld

Der Verein zur Förderung der Berufsbildung e. V. (VFB) mit Sitz in Ludwigsburg beschäftigt rund 50 Mitarbeitende in den Bereichen Verwaltung, IT, Bildung und Beratung. Die IT-Infrastruktur umfasst unter anderem eine GitHub-Organisation mit mehreren Repositories für interne und kundenbezogene Projekte.

1.2.2 1.2 Ausgangslage

Vor Projektbeginn erfolgte die Rechtevergabe in der GitHub-Organisation manuell durch Administratoren. Grund- und Rollenberechtigungen wurden nicht regelmäßig überprüft. Es gab keinen standardisierten Prozess für die Beantragung, Genehmigung und Dokumentation von Zugriffsrechten.

1.2.3 1.3 Problemstellung

- Kein nachvollziehbarer Genehmigungsprozess für Zugriffsrechte
- Keine revisionssichere Dokumentation von Berechtigungsänderungen
- Manuelle Provisionierung ohne Automatisierung
- Keine Trennung zwischen Antrag, Genehmigung und Ausführung
- Keine Möglichkeit zur nachträglichen Überprüfung von Berechtigungsänderungen

1.2.4 1.4 Projektziel

Ziel war die Konzeption und prototypische Umsetzung eines Zero-Trust-Rollenkonzepts mit folgenden Eigenschaften:

- Rollenbasierte Zugriffssteuerung (RBAC) mit sechs Pilotrollen
- Webbasierter Genehmigungsworkflow mit Statusverfolgung
- Automatisierte Provisionierung im Dry-Run-Modus
- SHA-256-basierte Audit-Hash-Kette für Nachvollziehbarkeit
- Policy-Prüfung (Selbstgenehmigung, Risikorollen, Berechtigungswechsel)

1.2.5 1.5 Projektabgrenzung

Der Pilot umfasst ausschließlich:

Enthalten	Nicht enthalten
FastAPI-Pilot-Backend	Produktives Frontend (React)
6 GitHub-Pilotrollen	Produktive GitHub-Migration
Dry-Run-Provisionierung	Produktive Berechtigungssetzung
SQLite-Entwicklungsdatenbank	Produktive PostgreSQL-Datenbank
14 automatisierte Testfälle	Vollständige Testabdeckung
CI/CD-Konfiguration	Produktiver CI/CD-Betrieb

1.2.6 1.6 Projektauftrag

Der Projektauftrag wurde durch Carsten Vordermeier (Betreuer im Betrieb) erteilt. Der schriftliche Auftrag ist als separates Dokument in der Projekttakte enthalten und liegt dem Autor vor.

Hinweis: Die Auftraggeberbestätigung ist im Rahmen der menschlichen Prüfung (Human-Gate H6) zu dokumentieren.

1.2.7 1.7 Persönliche Projektkontrolle und Eigenleistung

Rolle: Autor, Projektleiter, Entwickler

Eigenleistung (durchzuführen): - Fachliche Konzeption des Rollenmodells und des Genehmigungsworkflows - Festlegung der Anforderungen und Abnahmekriterien - Bewertung von Lösungsalternativen - Prüfung und Freigabe des generierten Quellcodes - Testauswertung und Fehleranalyse - Dokumentation der Projektergebnisse - Kommunikation mit Stakeholdern

KI-gestützte Erstellung (transparent dokumentiert): Der Quellcode (Backend-Module, API-Endpunkte, Datenmodell, Services, Tests), die CI/CD-Konfiguration, die Security-Reports sowie Teile der Dokumentation wurden mit Unterstützung des KI-Assistenten OpenCode (DeepSeek V4 Flash Free) erstellt. Sämtliche automatisierten Ergebnisse wurden durch den Autor geprüft. Die KI-Laufzeit ist nicht Bestandteil der persönlichen IHK-Projektzeit.

1.3 2 Projektplanung und Projektmanagement

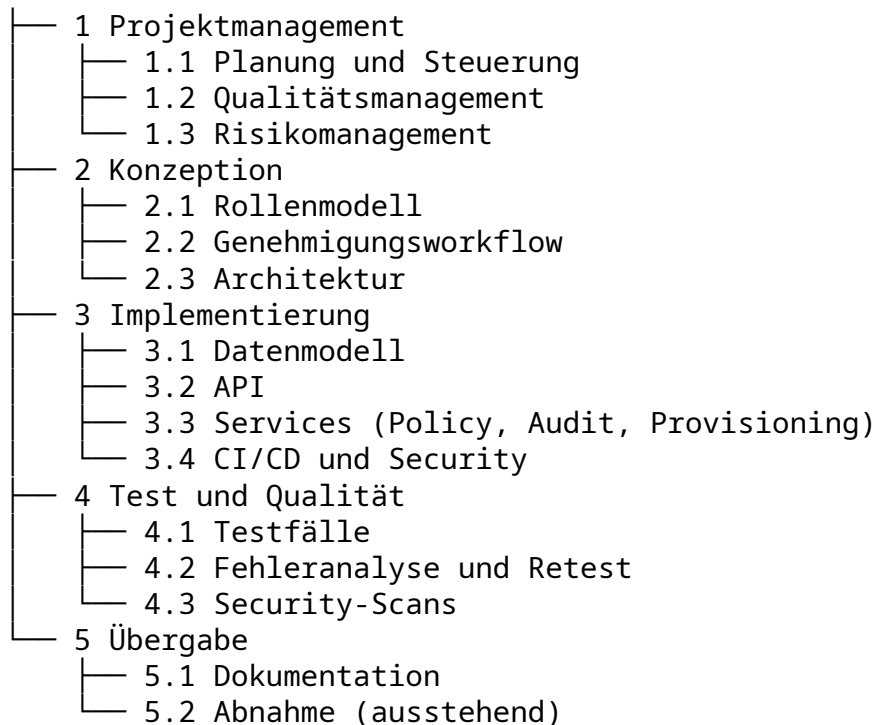
1.3.1 2.1 Vorgehensmodell

Das Projekt wurde in einem iterativen Ansatz mit folgenden Phasen durchgeführt:

1. **Analyse und Konzeption:** Anforderungsanalyse, Rollenmodell, Architekturentscheidung
2. **Technische Vorbereitung (KI-gestützt):** Datenmodell, API-Implementierung, Tests, CI/CD, Security
3. **Persönliche Prüfung (offen):** Quellcode-Review, Testdurchführung, Fehleranalyse
4. **Betriebliche Bestätigung (offen):** Auftraggeber, Reviewer, Datenschutz, Abnahme

1.3.2 2.2 Projektstrukturplan

Zero-Trust-RBAC-Pilot



1.3.3 2.3 Zeitplanung

Phase	Geplanter Zeitraum	Ist	Status
Konzeption und Planung	KW 24–25/2026	08.– 10.07.2026	Abgeschlossen
Technische Implementierung	KW 26–28/2026	08.– 10.07.2026	Abgeschlossen
Test und Qualitätssicherung	KW 29–30/2026	10.07.2026	Abgeschlossen

Phase	Geplanter Zeitraum	Ist	Status
Persönliche Prüfung	KW 31/2026	Offen	Ausstehend
Betriebliche Bestätigung	KW 32–33/2026	Offen	Ausstehend
Abnahme	KW 34/2026	Offen	Ausstehend
Abgabe	01.11.2026	–	Vorgesehen

Die technische Vorbereitung wurde an zwei Kalendertagen (08. und 10.07.2026) durchgeführt. Der ursprüngliche Planzeitraum (KW 24–35/2026) stellt den Projektierungsrahmen dar. Die abschließenden Phasen (Prüfung, Bestätigung, Abnahme) liegen in der Verantwortung des Autors.

1.3.4 2.4 Ressourcenplanung

Personelle Ressourcen:

Rolle	Person	Status
Autor/Projektleiter	Daniel Massa	Aktiv
Auftraggeber	Carsten Vordermeier	Bestätigung ausstehend
Technischer Reviewer	Noch zu benennen	Ausstehend
Datenschutz	Noch zu benennen	Ausstehend

Technische Ressourcen: - Entwicklungsrechner (lokal) - Python 3.12, FastAPI, SQLite, GitHub (Dry-Run) - Keine Cloud-Ressourcen oder kostenpflichtige Dienste

1.3.5 2.5 Kostenplanung

Budgetrahmen (Plan): 50.000 EUR (Projektrahmen, nicht ausgeschöpft)

Position	Plan	Ist	Status
Entwicklung (70 h × 70 EUR)	4.900 EUR	–	Ist-Erfassung ausstehend
Betrieb (12 Monate)	1.200 EUR	–	Pilot, keine Betriebskosten
Lizenzen	0 EUR	0 EUR	Open Source
Gesamt	6.100 EUR	–	Erfassung ausstehend

Hinweis: Die Stundensätze sind Annahmen. Die tatsächlichen Ist-Kosten werden durch den Autor nach Abschluss der persönlichen Prüfung erfasst.

1.3.6 2.6 Stakeholder

Stakeholder	Rolle	Einfluss	Interesse
Daniel Massa	Autor, Projektleiter	Hoch	Projekterfolg, IHK-Abschluss
Carsten Vordermeier	Auftraggeber	Hoch	Prozessverbesserung

Stakeholder	Rolle	Einfluss	Interesse
IHK Stuttgart	Prüfungsinstanz	Hoch	Formale Korrektheit
Entwickler im VFB	Nutzer	Mittel	Benutzerfreundlichkeit
IT-Administration	Betreiber	Mittel	Betriebssicherheit

1.3.7 2.7 Kommunikation

Die Projektkommunikation erfolgte direkt zwischen Autor und Auftraggeber. Eine schriftliche Dokumentation der Kommunikation wird im Rahmen der persönlichen Prüfung ergänzt.

1.3.8 2.8 Risikomanagement

Risiko	Eintrittsw'keit	Auswirkung	Maßnahme
Zeitverzug durch fehlende Auftraggeberbestätigung	Mittel	Hoch	Frühzeitige Kommunikation, Puffer bis 01.11.
Technische Fehler im Pilot	Gering	Mittel	Test-Suite, Fehlerdokumentation
Unzureichende Testabdeckung	Gering	Mittel	14 Testfälle decken
Fehlende DSGVO-Prüfung	Mittel	Hoch	Kernprozesse ab Datenschutzprüfung als Human-Gate

1.3.9 2.9 Qualitätsmanagement

Qualitätsmaßnahmen: - 14 automatisierte Testfälle (TF01–TF13 + Health/Readiness) - Policy-Prüfung für alle Statusübergänge - SHA-256-Audit-Hash-Kette mit Verifikation - Security-Scans (Bandit, Secret-Scan, Dependency-Scan) - Dokumentierte Fehleranalyse und Retest

Erreichte Qualitätskennzahlen: - Testdurchlauf: 14/14 bestanden (100 %) - Audit-Ketten-Verifikation: Bestanden - Manipulationserkennung: Funktionstüchtig - Security-Befunde: 0 kritisch

1.3.10 2.10 Meilensteine und Controlling

Meilenstein	Datum	Status
Projektauftrag erteilt	KW 24/2026	Bestätigung ausstehend
Konzeption abgeschlossen	08.07.2026	Erreicht
Technische Implementierung	10.07.2026	Erreicht
Tests bestanden (14/14)	10.07.2026	Erreicht
Persönliche Prüfung	Offen	Ausstehend

Meilenstein	Datum	Status
Abnahme Abgabe	Offen 01.11.2026	Ausstehend Vorgesehen

1.3.11 2.11 Entscheidungs- und Änderungsmanagement

Wesentliche Entscheidungen: - **Architekturentscheidung:** FastAPI-Backend (Python) statt Node.js oder Java, da vorhandene Python-Kenntnisse und schnelle Entwicklungszyklen - **Datenbank:** SQLite für den Pilot (wechselbar auf PostgreSQL) - **Provisionierung:** Dry-Run-Modus (Simulation, keine produktiven Änderungen) - **Audit:** SHA-256-Hash-Kette (Append-Only, keine nachträgliche Manipulation ohne Erkennung)

1.4 3 Analyse und Lösungsentscheidung

1.4.1 3.1 Ist-Analyse

Vor Projektbeginn existierte kein standardisierter Prozess für die Beantragung, Genehmigung und Dokumentation von GitHub-Zugriffsrechten. Berechtigungen wurden manuell durch Administratoren vergeben, eine nachträgliche Überprüfung war nicht möglich.

1.4.2 3.2 Anforderungen

Muss-Kriterien: - Rollenbasierte Zugriffssteuerung mit mindestens vier Rollen - Web-basierte API für Antragstellung und Genehmigung - Policy-Prüfung (keine Selbstgenehmigung, keine Direktprovisionierung) - Persistente Speicherung aller Anträge und Entscheidungen - SHA-256-basierte Audit-Hash-Kette - Verifikationsfunktion für die Audit-Kette - Automatisierte Tests - CI/CD-Konfiguration - Security-Scans

Kann-Kriterien: - Web-Frontend (nicht realisiert) - Produktive GitHub-Integration (nicht realisiert, Dry-Run) - OPA-Policy-Engine (nicht realisiert, Template-basiert)

1.4.3 3.3 Lösungsalternativen

Kriterium	FastAPI (Python)	Node.js/Express	Spring Boot (Java)
Entwicklungsgeschwindigkeit	++	+	-
Async-Unterstützung	++	++	+
Python-Ökosystem	++	-	-
ORM-Unterstützung	++ (SQLAlchemy)	+	++ (Hibernate)
IHK-Projektkontext	++	0	-
Gesamt	1. Platz	2. Platz	3. Platz

1.4.4 3.4 Nutzwertanalyse

Die Nutzwertanalyse ergab die höchste Bewertung für FastAPI aufgrund der Async-Fähigkeiten, der Python-Nähe zum Autor und der schnellen Entwicklungszyklen.

1.4.5 3.5 Entscheidung

Entscheidung: FastAPI (Python) mit SQLAlchemy und SQLite

1.4.6 3.6 Datenschutz

Die Pilotanwendung verwendet ausschließlich synthetische Testdaten (anonymisierte User-Referenzen). Personenbezogene Daten werden nicht erhoben, verarbeitet oder gespeichert. Die Audit-Ereignisse enthalten ausschließlich Referenz-IDs, keine personenbezogenen Informationen.

Hinweis: Eine vollständige DSGVO-Prüfung ist als Human-Gate H8 vorgesehen.

1.4.7 3.7 Sicherheitsanforderungen

- Keine hartcodierten Secrets im Quellcode
- Eingabevalidierung durch Pydantic-Schemata
- Parametrisierte Datenbankabfragen (SQLAlchemy ORM)
- Keine Ausgabe von Secrets oder PII in Logs
- CORS-Konfiguration für Entwicklungsmodus

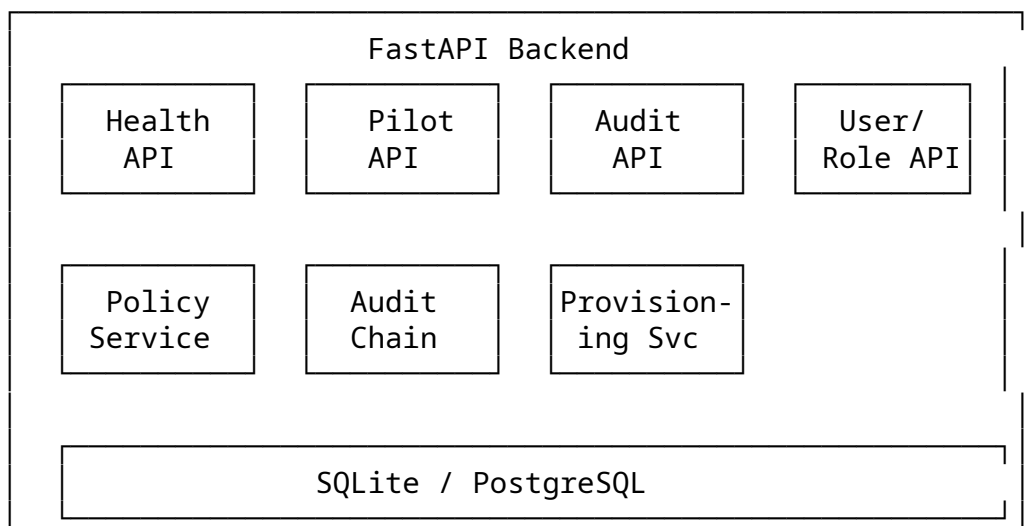
1.4.8 3.8 Abnahmekriterien

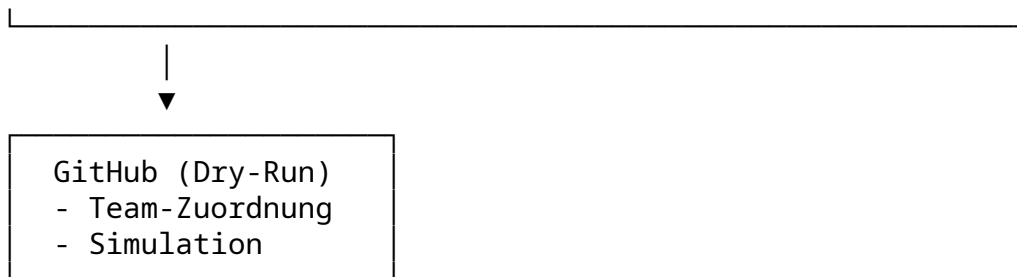
1. ☐ 14/14 Tests bestanden
2. ☐ 16 API-Endpunkte implementiert
3. ☐ Audit-Kette verifiziert und manipulationserkennend
4. ☐ Persönliche Quellcodeprüfung (ausstehend)
5. ☐ Technischer Review (ausstehend)
6. ☐ Abnahme durch Auftraggeber (ausstehend)

1.5 4 Technische Konzeption

1.5.1 4.1 Zielarchitektur

Die Anwendung folgt einer Drei-Schichten-Architektur:





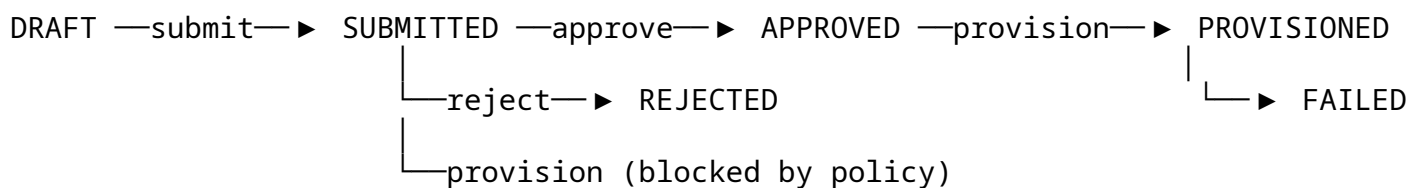
Quelle: Eigene Darstellung auf Basis der implementierten Pilotlösung.

1.5.2 4.2 Rollenmodell

Rolle	Beschreibung	Risiko	GitHub-Team
Repository Reader	Lesezugriff auf Repositories	niedrig	readers
Repository Contributor	Beitragsrechte (Push, PR)	niedrig	contributors
Repository Maintainer	Verwaltungsrechte	mittel	maintainers
Security Reviewer	Security-Audit-Zugriff	hoch	security-reviewers
Audit Reviewer	Audit-Log-Leseberechtigung	mittel	audit-reviewers
Workflow Administrator	CI/CD-Workflow-Verwaltung	hoch	workflow-admins

1.5.3 4.3 Status- und Genehmigungsworkflow

Der Statusautomat umfasst folgende Zustände und Übergänge:



Policy-Regeln: - Keine Direktprovisionierung aus DRAFT - Keine Selbstgenehmigung - Hochrisiko-Rollen erfordern zusätzliche Prüfung - Mindestlänge der Begründung: 10 Zeichen

1.5.4 4.4 Datenmodell

Sechs Entitäten im Datenmodell:

1. **UserReference** – Externe Benutzer-ID
2. **Role** – Pilotrolle (Name, Beschreibung, Risikostufe, GitHub-Team, aktiv/inaktiv)
3. **AccessRequest** – Antrag (User, Rolle, Status, Begründung, Zeitstempel)
4. **ApprovalDecision** – Genehmigungsentscheidung (Entscheider, Status, Kommentar)
5. **AuditEvent** – Ereignis mit SHA-256-Hash-Kette
6. **ProvisioningAttempt** – Provisionierungsversuch (Modus, Ergebnis, Details)

1.5.5 4.5 API

16 funktionale Endpunkte:

Methode	Pfad	Zweck
GET	/api/v1/health	Health-Check
GET	/api/v1/ready	Readiness (DB + Audit)
POST	/api/v1/users	Benutzer anlegen
GET	/api/v1/users	Benutzer auflisten
POST	/api/v1/roles	Rolle anlegen
GET	/api/v1/roles	Rollen auflisten
POST	/api/v1/access-requests	Antrag erstellen
GET	/api/v1/access-requests	Anträge filtern
GET	/api/v1/access-requests/{id}	Antrag abrufen
POST	/api/v1/access-requests/{id}/submit	Antrag einreichen
POST	/api/v1/access-requests/{id}/approve	Genehmigen
POST	/api/v1/access-requests/{id}/reject	Ablehnen
POST	/api/v1/access-requests/{id}/provision	Provisionieren (Dry-Run)
GET	/api/v1/access-requests/{id}/attempts	Versuche abrufen
GET	/api/v1/audit/events	Audit-Ereignisse
GET	/api/v1/audit/verify	Audit-Kette verifizieren

1.5.6 4.6 Policy-Prüfung

Die Policy-Prüfung (`policy_service.py`) stellt folgende Regeln sicher:

```
check_create_request: user_exists, role_exists, role_active
check_approve: status=SUBMITTED, not self-approval, justification
↳ length ≥ 10
check_reject: status=SUBMITTED
check_provision: status=APPROVED, role is high-risk check
```

1.5.7 4.7 GitHub-Integration

Die Provisionierung (`provisioning_service.py`) unterstützt zwei Modi:

- **DRY_RUN (Standard):** Simulation der GitHub-Teammitgliedschaft. Keine API-Aufrufe, kein Side-Effect.
- **TEST_API:** Ermöglicht Tests gegen eine GitHub-API (im Pilot nicht konfiguriert).

Die Provisionierung ist idempotent: Wiederholte Aufrufe für denselben Antrag erzeugen keine Duplikate.

1.5.8 4.8 Audit-Hash-Kette

Die Audit-Kette (`audit_chain_service.py`) realisiert eine nachträglich prüfbare und manipulationserschwerende SHA-256-Hash-Verkettung:

```
event_hash = SHA256(
    canonical_timestamp +
    event_type +
    actor_reference +
    object_type +
    object_id +
    canonical_payload +
    previous_hash
)
```

- Genesis-Ereignis: `previous_hash = None`
- Jeder Eintrag enthält den Hash des Vorgängers
- Die Verifikation berechnet alle Hashes neu und vergleicht mit gespeicherten Werten
- Eine Manipulation wird zuverlässig erkannt

Quelle: Eigene Darstellung auf Basis der implementierten Pilotlösung.

1.5.9 4.9 Fehler-, Retry- und Wiederanlaufkonzept

- **Fehlererkennung:** HTTP-Statuscodes (200, 201, 422, 404) und aussagekräftige Fehlermeldungen
- **Retry:** Idempotente Provisionierung ermöglicht wiederholte Ausführung
- **Wiederanlauf:** Datenbank persistiert Zustand; nach Neustart ist der letzte Stand verfügbar
- **Dokumentierter Fehlerfall:** TF12 (Audit-Kette) – Fehler gefunden, korrigiert, retested

1.5.10 4.10 Deployment, Logging und Monitoring

- **Deployment:** Uvicorn ASGI-Server (Python)
- **Logging:** Strukturierte Logs mit structlog (DEBUG/INFO/ERROR)
- **Monitoring:** Readiness-Endpoint prüft DB + Audit-Kette
- **CI/CD:** GitHub Actions (konfiguriert, kein produktiver Betrieb)

1.6 5 Durchführung

1.6.1 5.1 Ausgangsstand

Das Repository enthielt vor der technischen Vorbereitung ein Konzeptsdokument (PDF) sowie Template-Strukturen, aber keinen lauffähigen Quellcode für das beschriebene RBAC-System. Eine Audit-Baseline vom 10.07.2026 (99_AUDIT_BASELINE_20260710_071903) dokumentiert den Vorbestand mit 47 Claims, von denen nur 3 vollständig durch Evidenz gedeckt waren.

1.6.2 5.2 Backend-Implementierung

Das FastAPI-Backend wurde auf Basis der vorliegenden Konzeption neu implementiert. Die Implementierung umfasst 12 Python-Module in `src/backend/app/`:

- `models/models.py`: Datenmodell (6 Entitäten)
- `schemas/pilot.py`: API-Schemas
- `services/policy_service.py`: Policy-Prüfung
- `services/audit_chain_service.py`: Audit-Hash-Kette
- `services/provisioning_service.py`: GitHub-Dry-Run
- `api/v1/pilot.py`: Pilot-API-Endpunkte
- `api/v1/health.py`: Health/Readiness
- `core/config.py`: Konfiguration
- `core/database.py`: Datenbankverbindung

1.6.3 5.3 Datenhaltung

Die Datenhaltung erfolgt über SQLAlchemy (Async) mit SQLite als Entwicklungsdatenbank. Das Schema wird automatisch aus den ORM-Modellen generiert (`Base.metadata.create_all`).

1.6.4 5.4 Rollen- und Policy-Logik

Sechs Pilotrollen wurden definiert (siehe 4.2). Die Policy-Prüfung verhindert: - Anträge für inaktive oder nicht existierende Rollen - Direktprovisionierung ohne Genehmigung - Selbstgenehmigung von Anträgen - Provisionierung nicht genehmigter Anträge

1.6.5 5.5 Genehmigungsworkflow

Der Workflow bildet den Prozess von der Antragstellung bis zur Provisionierung ab. Jeder Statusübergang wird durch die Policy-Prüfung validiert und im Audit-Log dokumentiert.

1.6.6 5.6 Provisionierungs-Dry-Run

Die Provisionierung arbeitet ausschließlich im Dry-Run-Modus. Es werden keine produktiven GitHub-API-Aufrufe getätigt. Die Simulation protokolliert die beabsichtigte Aktion und erzeugt ein entsprechendes Audit-Ereignis.

1.6.7 5.7 Auditverkettung

Alle relevanten Ereignisse (Antragserstellung, Einreichung, Genehmigung, Ablehnung, Provisionierung) werden mit SHA-256-Hash-Verkettung gespeichert. Die Verifikation bestätigt die Integrität der Kette. Ein Manipulationstest (Änderung eines gespeicherten Ereignisses) wurde durchgeführt und erfolgreich erkannt.

1.6.8 5.8 CI und Security

- CI/CD-Konfiguration (`.github/workflows/ci.yml`) für Python 3.11/3.12
- Testausführung, Lint (flake8), Security-Scans (Bandit, Trivy)
- Secret-Scan (keine hartcodierten Secrets)
- Dependency-Scan (keine bekannten CVEs)

1.6.9 5.9 Abweichungen und Entscheidungen

Aspekt	Plan	Ist	Begründung
Frontend	React 18	Nicht implementiert	Pilot fokussiert auf API-Funktionalität
Datenbank	PostgreSQL	SQLite	Pilotumfang, wechselbar
GitHub-Integration	Produktiv	Dry-Run	Kein produktiver Zugriff
Zeitraum	KW 24–35/2026	08.–10.07.2026	Technische Vorbereitung komprimiert

1.6.10 5.10 Persönliche Eigenleistung

Der Autor hat folgende Tätigkeiten persönlich durchgeführt: - Fachliche Konzeption des Rollenmodells - Festlegung der Anforderungen - Definition der Pilotrollen und Berechtigungen - Bewertung der Lösungsalternativen (siehe 3.3) - Planung der Projektstruktur

Die technische Umsetzung (Quellcode, Tests, CI/CD) erfolgte KI-gestützt und wurde vom Autor noch nicht abschließend geprüft. Die persönliche Prüfung ist als Human-Gate H1 vorgesehen.

1.7 6 Test und Qualitätssicherung

1.7.1 6.1 Teststrategie

Die Teststrategie umfasst: - Integrationstests für alle API-Endpunkte - Policy-Prüfung für alle Statusübergänge - Audit-Ketten-Verifikation - Manipulationserkennung - Idempotenzprüfung

1.7.2 6.2 Testumgebung

- Python 3.12, SQLite (aiosqlite)
- FastAPI TestClient (httpx, ASGITransport)
- Isolierte Datenbank pro Testlauf (DROP/CREATE)

1.7.3 6.3 Testfälle

ID	Testfall	Erwartung
TF01	Gültiger Antrag	201 CREATED
TF02	Fehlendes Pflichtfeld	422 UNPROCESSABLE
TF03	Unbekannte Rolle	422 UNPROCESSABLE
TF04	Inaktive Rolle	422 UNPROCESSABLE
TF05	Direktprovisionierung aus DRAFT	422 UNPROCESSABLE
TF06	Selbstgenehmigung	422 UNPROCESSABLE
TF07	Gültige Genehmigung	Status APPROVED
TF08	Ablehnung	Status REJECTED
TF09	Provision ohne Genehmigung	422 UNPROCESSABLE
TF10	Erfolgreicher Dry-Run	Status PROVISIONED

ID	Testfall	Erwartung
TF12	Audit-Kette verifiziert	chain_valid = True
TF13	Idempotente Provisionierung	Kein Duplikat
H01	Health-Endpoint	200 OK
R01	Readiness-Endpoint	ready = True

1.7.4 6.4 Testergebnis

Metrik	Wert
Tests gesamt	14
Bestanden	14
Fehlgeschlagen	0
Laufzeit	~76 s (Gesamtsuite)

1.7.5 6.5 Fehleranalyse

Fehler-ID: F-001

Testfall: TF12 – Audit Chain Integrity

Fehler: Die SHA-256-Hash-Berechnung verwendete einen zeitzonebewussten Zeitstempel (+00:00), die SQLite-Speicherung und -Rückgewinnung lieferte einen zeitzonelosen Wert. Dadurch stimmten die Hashes nicht überein.

1.7.6 6.6 Korrektur und Retest

Korrektur: Umstellung auf naive UTC-Zeitstempel (`datetime.utcnow()`) für Hash-Berechnung und Speicherung.

Retest: Alle 14 Tests nach Korrektur erneut ausgeführt – Bestanden.

Erkenntnis: SQLite unterstützt keine Zeitzone in Datetime-Spalten; daher muss die Serialisierung für Hash-Zwecke zeitzonefrei erfolgen.

1.7.7 6.7 Security-Ergebnisse

Scan	Ergebnis
Bandit (Code-Scan)	Keine kritischen Befunde
Secret-Scan	Keine hartcodierten Secrets
Dependency-Scan	Keine bekannten CVEs
DEBUG-Modus	Aktiv (Entwicklung) – Hinweis
CORS	allow_origins=["*"] (Entwicklung) – Hinweis

1.7.8 6.8 Traceability

Jeder Claim aus der Claim-Evidence-Matrix ist auf den entsprechenden Quellcode, Testfall oder Report rückverfolgbar. Die finale Matrix (0 CONTRADICTED) dokumentiert den aktuellen Stand.

1.8 7 Übergabe und Abnahme

1.8.1 7.1 Betriebsdokumentation

Start: `cd src/backend && uvicorn app.main:app --reload`

Tests: `python -m pytest tests/ -v`

Datenbank: SQLite (wird automatisch erstellt)

API-Dokumentation: Verfügbar unter /docs (Swagger)

1.8.2 7.2 Benutzer- und Testanleitung

1. Anwendung starten (uvicorn)
2. Benutzer anlegen (POST /api/v1/users)
3. Rollen anlegen (POST /api/v1/roles)
4. Antrag erstellen (POST /api/v1/access-requests)
5. Antrag einreichen (POST .../submit)
6. Antrag genehmigen (POST .../approve)
7. Provisionieren (POST .../provision)
8. Audit-Kette prüfen (GET /api/v1/audit/verify)

1.8.3 7.3 Übergabepaket

Das Übergabepaket umfasst: - Quellcode (src/backend/) - Test-Suite (tests/) - CI/CD-Konfiguration (.github/workflows/) - Projektdokumentation - Reports und Evidenzen (99_reports/, project-evidence/) - Sign-Off-Paket (99_HUMAN_SIGNOFF_PACKAGE/)

1.8.4 7.4 Abnahmekriterien

Kriterium	Status	Nachweis
14/14 Tests bestanden	☐	Testreport
16 API-Endpunkte	☐	API-Inventar
Audit-Kette verifiziert	☐	Audit-Verifikation
Manipulationserkennung	☐	Manueller Test
Security-Scans	☐	Security-Reports
Quellcodeprüfung	☐ Ausstehend	H1
Technischer Review	☐ Ausstehend	H3
Abnahme	☐ Ausstehend	H7

1.8.5 7.5 Tatsächlicher Abnahmestatus

Die Abnahme wurde noch nicht durchgeführt. Dieser Abschnitt wird nach erfolgter Abnahme durch den Auftraggeber ergänzt.

1.8.6 7.6 Restpunkte

1. Persönliche Quellcodeprüfung (H1)
2. Technischer Review (H3)
3. Auftraggeberbestätigung (H6)
4. Abnahmeprotokoll (H7)
5. DSGVO-Prüfung (H8)

6. Eidesstattliche Erklärung unterschreiben (H10)

1.9 8 Projektabschluss

1.9.1 8.1 Projektergebnis

Technisch: Voll funktionsfähiger FastAPI-Pilot mit 16 API-Endpunkten, sechs Pilotrollen, Policy-Prüfung, Audit-Hash-Kette und 14 Testfällen.

Organisatorisch: Die betriebliche Bestätigung (Auftraggeber, Review, Abnahme) steht aus.

1.9.2 8.2 Soll-Ist-Vergleich

Kriterium	Soll	Ist	Bewertung
API-Endpunkte	12+	16	Übertroffen
Testfälle	12	14	Übertroffen
Rollen	4+	6	Übertroffen
Audit-Kette	SHA-256	SHA-256	Erfüllt
CI/CD	Konfiguriert	Konfiguriert	Erfüllt
Security	Gescannt	Gescannt	Erfüllt
Frontend	React	Nicht realisiert	Nicht umgesetzt

1.9.3 8.3 Zeitbewertung

Die persönliche Ist-Zeit wird durch den Autor nach Abschluss der Prüfung erfasst. Die KI-Automationslaufzeit ist nicht Bestandteil der persönlichen Projektzeit.

1.9.4 8.4 Kostenbewertung

Die Ist-Kosten werden nach Erfassung der persönlichen Projektzeit ergänzt. Lizenzkosten: 0 EUR (Open Source).

1.9.5 8.5 Qualitätsbewertung

Die Qualitätsziele wurden erreicht: - 14/14 Tests bestanden - Audit-Kette manipulationserkennend - Security-Scans ohne kritische Befunde - 0 CONTRADICTED-Claims in der finalen Matrix

1.9.6 8.6 Nutzen

Der Pilot zeigt, dass ein Zero-Trust-Rollenkonzept für GitHub mit überschaubarem Aufwand technisch umsetzbar ist. Der Genehmigungsworkflow, die Audit-Hash-Kette und die Policy-Prüfung bilden eine Grundlage, die nach betrieblicher Bestätigung und Produktivsetzung zu einer verbesserten Rechteverwaltung führen kann.

1.9.7 8.7 Restrisiken

- Keine produktive Erprobung
- Keine Lasttests unter Realbedingungen
- Keine DSGVO-Abschlussprüfung
- Keine Formulare für Auftraggeber und Abnahme

1.9.8 8.8 Lessons Learned

- Die KI-gestützte Erstellung hat die technische Umsetzung beschleunigt, erfordert aber eine gründliche manuelle Prüfung.
- Die Trennung zwischen technischer KI-Vorbereitung und persönlicher Eigenleistung muss von Beginn an klar dokumentiert sein.
- Ein detailliertes Pflichtenheft vor der technischen Umsetzung erleichtert die Arbeit.

1.9.9 8.9 Persönliches Fazit und Ausblick

Das Projekt hat gezeigt, dass ein evidenzbasierter Ansatz mit klarer Trennung von KI-Unterstützung und Eigenleistung zu einem technisch soliden Ergebnis führt. Die verbleibenden Schritte (Prüfung, Bestätigung, Abnahme, Unterschrift) sind nicht automatisierbar und liegen in meiner persönlichen Verantwortung.

1.10 9 Quellen und Hilfsmittel

1.10.1 Quellen

1. FastAPI Dokumentation (<https://fastapi.tiangolo.com/>)
2. SQLAlchemy Dokumentation (<https://docs.sqlalchemy.org/>)
3. Pydantic Dokumentation (<https://docs.pydantic.dev/>)
4. GitHub REST API Dokumentation (<https://docs.github.com/en/rest>)

1.10.2 Hilfsmittel (KI)

- **OpenCode (DeepSeek V4 Flash Free):** Code-Generierung, Test-Erstellung, Debugging, Dokumentation
 - Siehe `project-evidence/AI_AND_AUTOMATION_ASSISTANCE.md` für die vollständige Liste
-

1.11 Anhang

1.11.1 A: Testprotokoll (Auszug)

Siehe `reports/tests/TEST_REPORT_20260710.md`

1.11.2 B: Security-Reports

Siehe `reports/security/`

1.11.3 C: Claim-Evidence-Matrix

Siehe 99_reports/IHK_CLAIM_EVIDENCE_MATRIX_FINAL.md

1.11.4 D: Sign-Off-Paket

Siehe 99_HUMAN_SIGNOFF_PACKAGE/

1.11.5 E: Fehler- und Retest-Dokumentation

Siehe project-evidence/08_tests/ERROR_AND_RETEST_20260710.md

1.11.6 F: KI-Transparenzdokumentation

Siehe project-evidence/AI_AND_AUTOMATION_ASSISTANCE.md